

LAPORAN AKHIR
PRATIUM MIKROKONTROLER
PROJECT MEMBUAT PAKAN BINATANG



Oleh :

NAMA	: Satria Bhukti Akhisnu (22/505927/SV/21999) Muhandis Lawdza'I Putra Sanjaya (22/496604/SV/20992) Rizq Syandana Dwi Anggara (22/496554/SV/20979)
NIM	: 22/496604/SV/20992
KELAS	: TRE B1
HARI, TANGGAL	: Senin, 27 November 2023
DOSEN PENGAMPU	: Ir. Muhammad Arrofiq,S.T.,M.T.Ph.D

LABORATORIUM MIKROKONTROLER
PROGRAM SARJANA TERAPAN TEKNOLOGI REKAYASA ELEKTRO
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
2023

A. TUJUAN

Setelah melaksanakan Praktikum, mahasiswa mampu :

1. Mahasiswa mampu memahami dan membuat program Serial Komunikasi pada Arduino
2. Mahasiswa mampu merangkai rangkaian komponen Serial Komunikasi
3. Mahasiswa mampu mengontrol kecepatan motor dan suhu dengan menggunakan komponen sensor pada arduino mega 2560

B. ALAT DAN BAHAN

Alat dan bahan yang dibutuhkan untuk melaksanakan praktikum unit ini adalah :

1. komputer/notebook.
2. Papan pelatihan Arduino
3. Sambungan internet
4. Alat tulis
5. Kabel *jumper*
6. Kamera
7. (Disesuaikan dengan kebutuhan praktikum)

C. LANGKAH KERJA

Urutan-urutan kerja yang dilakukan dalam praktikum ini memiliki susunan sebagai berikut :

1. Identifikasikan bagian-bagian yang ada pada papan arduino
2. Carilah fungsi-fungsi sesuai dengan modul yang ada pada papan arduino
3. Jalankan software Arduino sesuai dengan petunjuk project
4. Pasang papan Arduino dan sambungkan dengan laptop/pc lalu jalankan dengan software Arduino IDE sesuai dengan project praktikum.

D. DATA DAN PEMBAHASAN

PROJECT :

Dibutuhkan sebuah piranti untuk mengaktifkan dan menonaktifkan feeder pakan Binatang terjadwal dan rutin setiap hari selama 24/7

deskripsi sistem :

sistem yang dibangun akan mengelola pemberi pakan (feeder) terdapat 4 buah kandang ternak yang harus dikelola. Feeder 1 disimulasikan dengan LED 1, feeder 2 disimulasikan dengan LED 2, feeder 3 disimulasikan dengan LED 3 dan feeder 4 disimulasikan dengan LED 4

Aktif dan tidaknya feeder ditandai dengan nyala/matinya LED

dibutuhkan 4 buah tombol untuk merealisasikan fungsi atur , up, down, dan enter

tampilan pertama : jika tombol atur ditekan, muncul : pilihan feeder yang akan diset

dengan menekan tombol up atau down. Pilihan Feeder 1, Feeder 2, Feeder 3, Feeder 4, Cancel. Jika terpilih cancel , kembali ke menu utama.

setelah salah satu feeder dipilih,

Menu kedua adalah setting waktu on set jam gunakan tombol UP untuk menaikkan angka, tombol down untuk menurunkan angka, setelah sesuai, tekan tombol ENTER.

Setelah itu set menit gunakan tombol up untuk menaikkan angka, tombol DOWN untuk menurunkan angka, setelah sesuai, tekan tombol ENTER.

Setelah itu setting waktu off

SET JAM. Gunakan tombol UP untuk menaikkan angka, tombol DOWN untuk menurunkan angka, setelah sesuai, tekan tombol, ENTER.

Setelah itu SET MENIT. Gunakan tombol UP untuk menaikkan angka, tombol DOWN untuk menurunkan angka, setelah sesuai, tekan tombol ENTER

Coding Program :

```
#include <Wire.h>
#include <LiquidCrystal.h>
#include <RTCLib.h>

RTC_DS1307 rtc;
LiquidCrystal lcd(36, 34, 32, 30, 28, 26);

const int buttonENTER = 40;
const int buttonUP = 42;
const int buttonDOWN = 44;
const int buttonATUR = 46;

const int LED1 = 31;
const int LED2 = 33;
const int LED3 = 35;
const int LED4 = 37;

enum State {
    MAIN_MENU,
    FEEDER_SELECTION,
    SET_ON_HOUR,
    SET_ON_MINUTE,
    SET_OFF_HOUR,
    SET_OFF_MINUTE
};

String feeders[] = {"Feeder 1", "Feeder 2", "Feeder 3", "Feeder 4", "Cancel"};
const int TOTAL_FEEDERS = 4;

// Variabel untuk menyimpan setting waktu setiap feeder
struct FeederSettings {
    int onHour;
    int onMinute;
    int offHour;
    int offMinute;
    bool isSetting; // Menandakan apakah waktu sedang diatur
};

FeederSettings feederSettings[TOTAL_FEEDERS];

int selectedFeeder = 0;
State currentState = MAIN_MENU;

void setup() {
```

```

Wire.begin();
rtc.begin();

lcd.begin(16, 2);

pinMode(buttonATUR, INPUT_PULLUP);
pinMode(buttonUP, INPUT_PULLUP);
pinMode(buttonDOWN, INPUT_PULLUP);
pinMode(buttonENTER, INPUT_PULLUP);

pinMode(LED1, OUTPUT);
pinMode(LED2, OUTPUT);
pinMode(LED3, OUTPUT);
pinMode(LED4, OUTPUT);

lcd.setCursor(0, 0);
lcd.print("Select Feeder");
}

void showFeederSelection() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print(feeders[selectedFeeder]);

    lcd.setCursor(0, 1);
    lcd.print("Use UP/DOWN");
}

void showSettingTime(const char *settingType, int value) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print(settingType);

    lcd.setCursor(1, 1);
    lcd.print(" ");
    lcd.print(value);
}

void showCurrentTime(DateTime now) {
    if (currentState == MAIN_MENU) {
        lcd.setCursor(0, 1);
        lcd.print(" "); // Clear the second line
        lcd.setCursor(0, 1);
        lcd.print(now.hour() < 10 ? "0" : "");
        lcd.print(now.hour());
        lcd.print(":");
    }
}

```

```

    lcd.print(now.minute() < 10 ? "0" : "");
    lcd.print(now.minute());
    lcd.print(":");
    lcd.print(now.second() < 10 ? "0" : "");
    lcd.print(now.second());
}
}

```

```

void controlLED(int feeder, bool state) {
    // Tambahkan kondisi untuk memeriksa apakah waktu sedang diatur
    if (!feederSettings[feeder].isSetting) {
        switch (feeder) {
            case 0:
                digitalWrite(LED1, state ? HIGH : LOW);
                break;
            case 1:
                digitalWrite(LED2, state ? HIGH : LOW);
                break;
            case 2:
                digitalWrite(LED3, state ? HIGH : LOW);
                break;
            case 3:
                digitalWrite(LED4, state ? HIGH : LOW);
                break;
        }
    }
}

```

```

void loop() {
    DateTime now = rtc.now();
    switch (currentState) {
        case MAIN_MENU:
            if (digitalRead(buttonENTER) == LOW) {
                while (digitalRead(buttonENTER) == LOW) {
                    delay(10);
                }
                currentState = FEEDER_SELECTION;
                showFeederSelection();
                delay(1000);
            }
            break;
        case FEEDER_SELECTION:
            if (digitalRead(buttonUP) == LOW) {
                while (digitalRead(buttonUP) == LOW) {
                    delay(10);
                }
            }
    }
}

```

```

    selectedFeeder++;
    if (selectedFeeder >= TOTAL_FEEDERS + 1) {
        selectedFeeder = 0;
    }
    showFeederSelection();
    delay(500);
}
if (digitalRead(buttonDOWN) == LOW) {
    while (digitalRead(buttonDOWN) == LOW) {
        delay(10);
    }
    selectedFeeder--;
    if (selectedFeeder < 0) {
        selectedFeeder = TOTAL_FEEDERS;
    }
    showFeederSelection();
    delay(500);
}
if (digitalRead(buttonATUR) == LOW) {
    while (digitalRead(buttonATUR) == LOW) {
        delay(10);
    }
    if (feeders[selectedFeeder] == "Cancel") {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Select Feeder");
        currentState = MAIN_MENU;
        delay(1000);
    } else {
        currentState = SET_ON_HOUR;
        // Tandai bahwa waktu sedang diatur
        feederSettings[selectedFeeder].isSetting = true;
        showSettingTime("Set ON Hour", feederSettings[selectedFeeder].onHour);
        delay(1000);
    }
}
}
break;

case SET_ON_HOUR:
    if (digitalRead(buttonUP) == LOW) {
        while (digitalRead(buttonUP) == LOW) {
            delay(10);
        }
        feederSettings[selectedFeeder].onHour = (feederSettings[selectedFeeder].onHour + 1) % 24;
        showSettingTime("Set ON Hour", feederSettings[selectedFeeder].onHour);
        delay(500);
    }
}

```

```

    }

    if (digitalRead(buttonDOWN) == LOW) {
        while (digitalRead(buttonDOWN) == LOW) {
            delay(10);
        }
        feederSettings[selectedFeeder].onHour = (feederSettings[selectedFeeder].onHour + 23) %
24;
        showSettingTime("Set ON Hour", feederSettings[selectedFeeder].onHour);
        delay(500);
    }
    if (digitalRead(buttonENTER) == LOW) {
        while (digitalRead(buttonENTER) == LOW) {
            delay(10);
        }
        currentState = SET_ON_MINUTE;
        showSettingTime("Set ON Min", feederSettings[selectedFeeder].onMinute);
        delay(1000);
    }
    break;

case SET_ON_MINUTE:
    if (digitalRead(buttonUP) == LOW) {
        while (digitalRead(buttonUP) == LOW) {
            delay(10);
        }
        feederSettings[selectedFeeder].onMinute = (feederSettings[selectedFeeder].onMinute + 1) %
60;
        showSettingTime("Set ON Min", feederSettings[selectedFeeder].onMinute);
        delay(500);
    }
    if (digitalRead(buttonDOWN) == LOW) {
        while (digitalRead(buttonDOWN) == LOW) {
            delay(10);
        }
        feederSettings[selectedFeeder].onMinute = (feederSettings[selectedFeeder].onMinute +
59) % 60;
        showSettingTime("Set ON Min", feederSettings[selectedFeeder].onMinute);
        delay(500);
    }
    if (digitalRead(buttonENTER) == LOW) {
        while (digitalRead(buttonENTER) == LOW) {
            delay(10);
        }
        currentState = SET_OFF_HOUR;
        showSettingTime("Set OFF Hour", feederSettings[selectedFeeder].offHour);
    }

```



```

    delay(1000);
}
break;

case SET_OFF_HOUR:
    if (digitalRead(buttonUP) == LOW) {
        while (digitalRead(buttonUP) == LOW) {
            delay(10);
        }
        feederSettings[selectedFeeder].offHour = (feederSettings[selectedFeeder].offHour + 1) %
24;
        showSettingTime("Set OFF Hour", feederSettings[selectedFeeder].offHour);
        delay(500);
    }

    if (digitalRead(buttonDOWN) == LOW) {
        while (digitalRead(buttonDOWN) == LOW) {
            delay(10);
        }
        feederSettings[selectedFeeder].offHour = (feederSettings[selectedFeeder].offHour + 23) %
24;
        showSettingTime("Set OFF Hour", feederSettings[selectedFeeder].offHour);
        delay(500);
    }
    if (digitalRead(buttonENTER) == LOW) {
        while (digitalRead(buttonENTER) == LOW) {
            delay(10);
        }
        currentState = SET_OFF_MINUTE;
        showSettingTime("Set OFF Min", feederSettings[selectedFeeder].offMinute);
        delay(1000);
    }
    break;

case SET_OFF_MINUTE:
    if (digitalRead(buttonUP) == LOW) {
        while (digitalRead(buttonUP) == LOW) {
            delay(10);
        }
        feederSettings[selectedFeeder].offMinute = (feederSettings[selectedFeeder].offMinute +
1) % 60;
        showSettingTime("Set OFF Min", feederSettings[selectedFeeder].offMinute);
        delay(500);
    }
    if (digitalRead(buttonDOWN) == LOW) {
        while (digitalRead(buttonDOWN) == LOW) {

```

```

        delay(10);
    }
    feederSettings[selectedFeeder].offMinute = (feederSettings[selectedFeeder].offMinute +
59) % 60;
    showSettingTime("Set OFF Min", feederSettings[selectedFeeder].offMinute);
    delay(500);
}
if (digitalRead(buttonENTER) == LOW) {
    while (digitalRead(buttonENTER) == LOW) {
        delay(10);
    }
    currentState = MAIN_MENU;
    showFeederSelection();
    // Tandai bahwa waktu selesai diatur
    feederSettings[selectedFeeder].isSetting = false;
    delay(1000);
}
break;
}

// Hanya menampilkan waktu saat berada di menu utama
showCurrentTime(now);

// Tampilkan LED dan waktu saat bukan dalam mode pemilihan feeder
if (currentState != FEEDER_SELECTION) {
    // Iterasi melalui setiap feeder
    for (int i = 0; i < TOTAL_FEEDERS; ++i) {
        // Matikan LED saat waktu set off tercapai
        if (now.hour() == feederSettings[i].offHour && now.minute() ==
feederSettings[i].offMinute) {
            controlLED(i, false);
        }
        // Hidupkan LED saat waktu set on tercapai
        else if (now.hour() == feederSettings[i].onHour && now.minute() ==
feederSettings[i].onMinute) {
            controlLED(i, true);
        } else {
            controlLED(i, false); // Matikan LED jika tidak dalam rentang waktu set on
        }
    }
}

delay(100);
}

```

Definisi Fungsi Program Arduino :

1. Penggunaan Library dan Inisialisasi Hardware

Program menggunakan tiga library, yaitu Wire untuk komunikasi I2C, LiquidCrystal untuk mengendalikan LCD, dan RTClib untuk berinteraksi dengan modul RTC DS1307. Objek RTC_DS1307 dan LiquidCrystal diinisialisasi untuk digunakan dalam program.

2. Definisi Tombol dan LED

Pin untuk tombol navigasi (ENTER, UP, DOWN, ATUR) dan LED (LED1, LED2, LED3, LED4) didefinisikan. Tombol digunakan untuk navigasi dalam menu dan konfirmasi, sementara LED merepresentasikan status pengumpanan pakan.

3. State Machine dan Menu

State machine diimplementasikan menggunakan enum State, yang mencakup MAIN_MENU, FEEDER_SELECTION, SET_ON_HOUR, SET_ON_MINUTE, SET_OFF_HOUR, dan SET_OFF_MINUTE. Program beroperasi melalui berbagai state untuk mengelola proses pemilihan pengumpanan dan pengaturan waktu.

4. Struktur Data untuk Pengaturan Waktu

Struktur FeederSettings digunakan untuk menyimpan pengaturan waktu setiap pengumpanan. Informasi waktu nyalakan (on) dan matikan (off) disimpan, serta variabel isSetting untuk menandakan apakah sedang dalam proses pengaturan waktu.

5. Fungsi-fungsi Tampilan dan Kontrol

Beberapa fungsi digunakan untuk menampilkan informasi di LCD dan mengendalikan LED. Ini termasuk fungsi showFeederSelection(), showSettingTime(), showCurrentTime(), dan controlLED().

6. Loop Utama

Loop utama berisi logika kontrol utama berdasarkan state mesin keadaan. Sistem berperilaku sesuai dengan state saat ini, termasuk pemilihan pengumpanan, pengaturan waktu, dan pemantauan waktu real-time.

7. Fitur Tambahan

Program menggunakan modul RTC DS1307 untuk mendapatkan waktu real-time. Pengaturan waktu dilakukan melalui tombol-tombol navigasi dan dikonfirmasi dengan tombol "ATUR". LED mewakili status pengumpanan, menyala saat waktu pengumpanan aktif dan mati saat waktu pengumpanan tidak aktif

Cara Kerja Program :

Program di atas adalah sebuah program untuk mengontrol waktu pemberian makanan pada beberapa feeder menggunakan Arduino. Program ini menggunakan beberapa library, yaitu Wire, LiquidCrystal, dan RTClib. Program ini juga menggunakan beberapa variabel konstan untuk menentukan pin yang terhubung dengan tombol, LED, dan LCD. Program ini memiliki beberapa state, seperti MAIN_MENU, FEEDER_SELECTION, SET_ON_HOUR, SET_ON_MINUTE, SET_OFF_HOUR, dan SET_OFF_MINUTE. Setiap feeder memiliki setting waktu yang disimpan dalam struktur FeederSettings. Program ini juga mengontrol LED berdasarkan waktu yang telah diatur untuk setiap feeder. Selain itu, program ini menggunakan beberapa tombol untuk memilih feeder dan mengatur waktu pemberian makanan. Ketika tombol ditekan, program akan memperbarui state dan menampilkan waktu serta mengontrol LED berdasarkan waktu yang telah diatur. Program ini juga menggunakan RTC untuk mendapatkan waktu sekarang dan mengatur waktu pemberian makanan pada setiap feeder berdasarkan waktu yang telah diatur

Jika tombol 1 (Atur) ditekan, maka akan muncul : Pilihan feeder yang akan diset.

Dengan menekan tombol UP atau Down (2/3) maka pilihan Feeder 1, Feeder 2, Feeder 3, dan feeder 4 maka feeder tersebut akan cancel dan Kembali ke menu utama.

Saat sudah memilih feeder maka tekan tombol enter (4) setelah salah satu feeder dipilih maka akan menuju ke menu kedua yaitu :

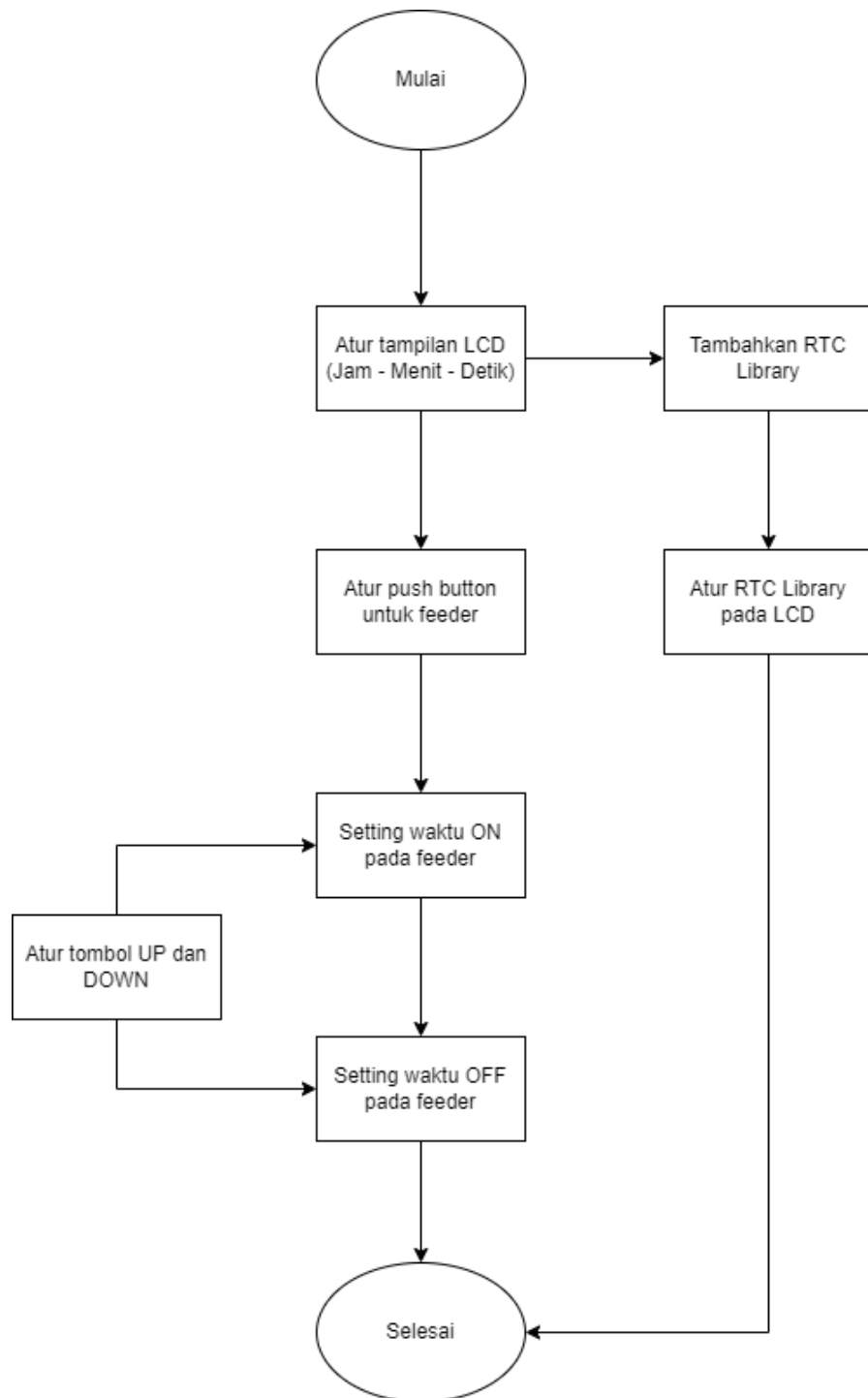
1. Setting Waktu On

Set Jam digunakan dengan tombol Up (2) untuk menaikkan angka, dan setelah sesuai, tekan tombol Enter. Setelah set jam maka selanjutnya yaitu akan menampilkan Set Menit. Gunakan tombol Up (2) untuk menaikkan angka, tombol Down untuk menurunkan angka, setelah sesuai, tekan tombol Enter (4).

2. Setting Waktu Off

Set Jam digunakan tombol Up (4) untuk menaikkan angka, tombol Down untuk menurunkan angka, setelah sesuai, tekan tombol Enter (4). Setelah itu Set Menit digunakan tombol Up untuk menaikkan angka, tombol Down untuk menurunkan angka, setelah sesuai, amak tekan tombol Enter.

Flow Chart :



E. KESIMPULAN

Kode ini menyusun berbagai konsep seperti state machine, input handling, dan kontrol waktu untuk menciptakan sistem pakan otomatis yang dapat diatur.

Struktur data digunakan untuk menyimpan pengaturan waktu setiap pengumpan, dan kode bersifat modular untuk mempermudah pemeliharaan dan perluasan fitur di masa mendatang.